Лекция 8. Параллельные численные алгоритмы

Рассмотрим применение параллельных алгоритмов при решении численных задач. Начав с двух классов параллельных алгоритмов умножения матриц, перейдем к решению систем линейных уравнений.

Умножение матриц в параллельных сетях

Один из способов параллельного матричного умножения пользуется сетью, размер которой связан с величиной матриц. Для умножения I x J-матрицы на J x K-матрицу потребуется двумерная решетка процессоров размером I x K, строчки которой соответствуют строчкам первой матрицы, а столбцы – столбцам второй.

Элементы первой матрицы будут поочередно поступать в строчки сети. Первая строчка попадет туда при первом проходе, вторая – при втором, и т.д.

Аналогичные действия будут выполняться с элементами второй матрицы и колонками сети, начиная с первой колонки матрицы. Запаздывание с поступлением элементов последующих строк и столбцов оказывается как раз таким, что числа, которые должны быть перемножены каждым процессором, поступают в него одновременно. Процессор умножает два числа, поступающие в него в очередном цикле, и прибавляет результат к текущему значению. В конце концов в каждом процессоре окажется одно результирующее значение. Этапы умножения изображены на рис. 1 – 9.

Анализ

Этот алгоритм выполняет умножение 2 x 3-матрицы на 3 x 4-матрицу за семь шагов. Для подсчета числа шагов в общем случае можно вывести формулу. Рассмотрим сначала сам процесс умножения. Один из способов подсчета числа операций состоит в том, чтобы вычислить число шагов, необходимых для полного прогона по сети последнего элемента последней строки первой матрицы. То же самое вычисление работает и для последнего элемента последнего столбца второй матрицы.

Строки первой матрицы сдвигаются на один столбец при каждом цикле. Если в первой матрице I строчек и J столбцов, то сколько циклов потребуется, чтобы

последний элемент первой строки покинул сеть? Каждый цикл добавляет в сеть одно число, а первое число поступает в сеть на первом цикле, последний элемент первой строки будет находиться в сети еще $J - 1$ цикл. Сколько дополнительных циклов потребуется, чтобы этот элемент покинул сеть, когда работа с ним закончится? Это число нужно умножить на одно значение из каждого столбца J x K-матрицы, поэтому на то, чтобы оно покинуло сеть, требуется K циклов. Из-за задержки очередной строки и очередного столбца теперь, когда стало ясно, как обрабатывается первая строка, необходимо понять, что происходит с последней строкой.

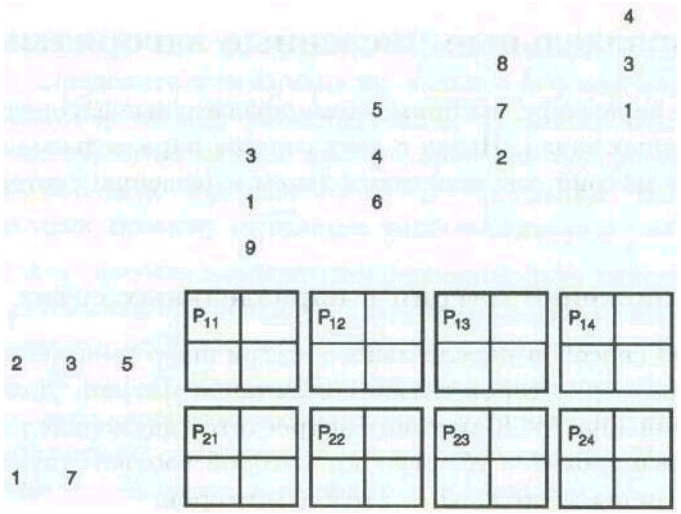


Рис. 1. Исходная решетчатая конфигурация сети

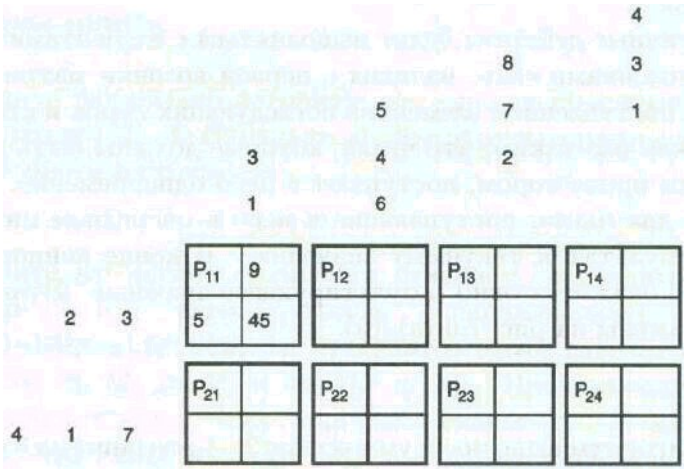


Рис. 2. Процессор P_{11} перемножает первые два поступивших значения и записывает их в своих регистрах

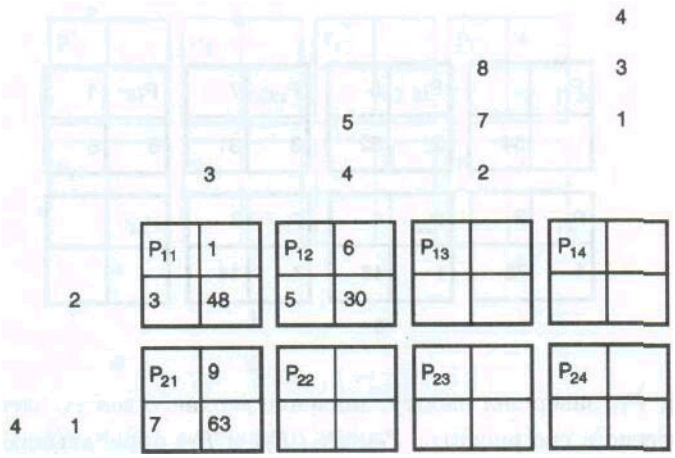


Рис. 3. Процессор P_{11} перемножает следующие два значения и прибавляет произведение к значению регистра; процессоры P_{12} и P_{21} перемножают первые поступившие к ним пары значений

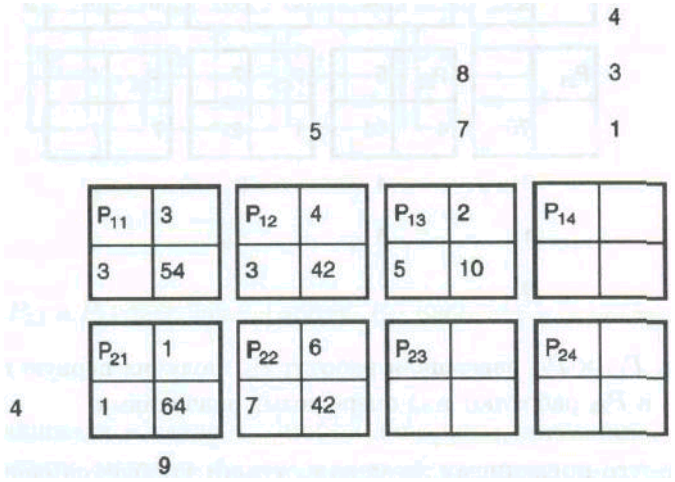


Рис. 4. Процессоры P_{11}, P_{12} и P_{21} обрабатывают очередные пары чисел, а P_{13} и P_{22} присоединяются к процессу обработки

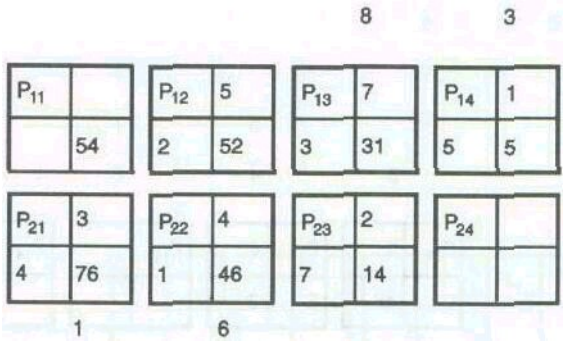


Рис. 5. Процессор P_{11} завершил работу, значение верхнего левого элемента произведения занесено в его регистр; P_{12}, P_{21}, P_{13} и P_{22} обрабатывают очередные пары чисел, а P_{14} и P_{23} присоединяются к процессу обработки

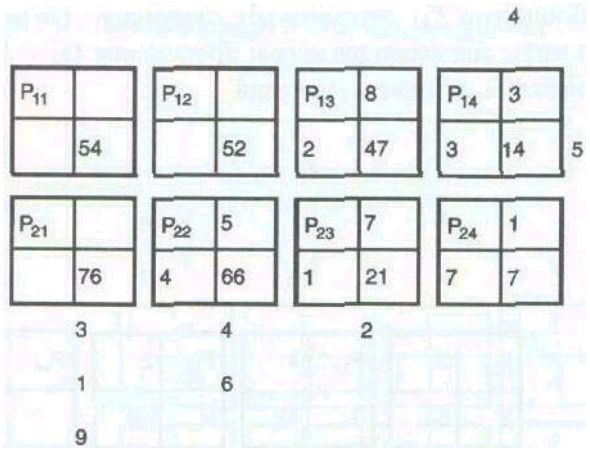


Рис. 6. Процессоры P_{12} и P_{21} завершили работу; P_{24} получил первую пару чисел, а P_{13}, P_{22}, P_{14} и P_{23} работают над очередными значениями

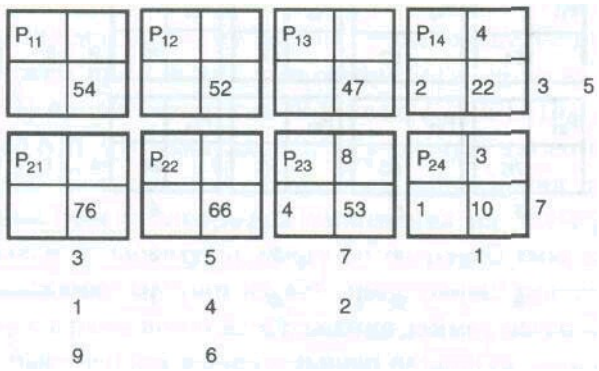


Рис. 7. Процессоры P_{13} и P_{22} завершили работу, а P_{14}, P_{23} и P_{24} работают над очередными значениями

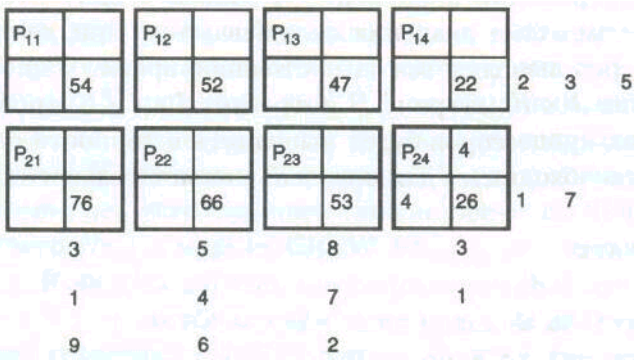


Рис. 8. Процессоры P_{14} и P_{23} завершили работу, P_{24} работает над последними двумя значениями

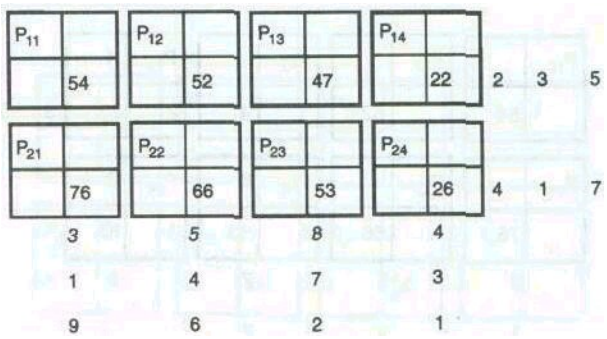


Рис. 9. Работа завершена и в процессорах записан результат

Как уже отмечалось, обработка очередной строки первой матрицы начинается первым циклом после начала обработки предыдущей строки. Вторая строка начинает обрабатываться на втором цикле, третья – на третьем, последняя – на цикле с номером I. Мы уже говорили о том, что последнему элементу строки требуется еще $J - 1$ цикл, чтобы попасть в сеть, и еще K циклов нужно ему, чтобы выйти из сети. Значит весь процесс завершится за $I + J + K - 1$ цикл. Время выполнения сетевого умножения поэтому имеет порядок $O(N)$, где $N = \max(I, J, K)$. Число используемых процессоров равно $O(N^2)$, поэтому стоимость параллельного алгоритма равна $O(N^3)$, т. е. она совпадает со стоимостью стандартного алгоритма матричного умножения. Основное его достоинство, однако, в том, что время его выполнения значительно меньше, чем у любого из последовательных алгоритмов.

Любой алгоритм, интенсивно использующий умножение матриц, будет работать на двумерной сети гораздо быстрее. Например, при свертке, где происходит умножение 5 x 5-матрицы на всякий 5 x 5-участок изображения размером 512 x 512, стандартный последовательный алгоритм умножения матриц потребовал бы 32 258 800 умножений, каждое из которых составляет отдельный цикл – всего 32 258 800 циклов. На сети размером 5 x 5 (25 процессоров) число умножений было бы тем же самым, однако все действия были бы совершены за 3 612 896 циклов – почти 90%-ая экономия времени.

Умножение матриц в модели *CRCW PRAM*

Параллельное умножение матриц в комбинированной модели *CRCW PRAM*, в которой при конкурентной записи в одну и ту же ячейку памяти записываемые значения складываются, при достаточном числе процессоров выполняется за

постоянное время. Умножение I x J-матрицы A на J x K-матрицу B потребует $I * J * K$ процессоров. Каждый из этих процессоров будет выполнять в точности одно из $O(N^3)$ умножений, необходимых для получения окончательного результата:

```
Parallel Start
    for x = 1 to I do
        for y = 1 to J do
            for z = 1 to K do
                P[x, y, z] вычисляет A[x, y] * B[y, z]
                        и записывает результат в M[x, z]
            end for z
        end for y
    end for x
Parallel End
```

Анализ

Как уже упоминалось, каждый из процессоров производит одно умножение и записывает его результат в нужную ячейку памяти. На это уходит один цикл. В каждую ячейку памяти, предназначенную для хранения результата, конкурентную запись производят J процессоров. Мы потребовали, чтобы эта конкурентная модель записи складывала все записываемые значения, поэтому другая часть стандартного алгоритма умножения матриц – сложение – выполняется при записи.

Этот алгоритм производит умножение матриц за время $O(1)$ и использует $O(N^3)$ процессоров, где $N = \max(I, J, K)$. Поэтому его стоимость равна $O(N^3)$ – такая же, как и у стандартного последовательного алгоритма. Сокращение же времени выполнения даже еще более принципиальное, чем у алгоритма умножения на двумерной решетке. Так, возвращаясь к нашему примеру со сверткой, видно, что однократное умножение матриц на 125 процессорах происходит за один цикл. Поэтому свертка всего изображения займет всего 258 064 циклов, что в 125 раз быстрее последовательного варианта.

Решение систем линейных уравнений алгоритмом *SIMD*

В последовательном алгоритме решения системы N линейных уравнений с N неизвестными методом Гаусса - Жордана систему линейных уравнений записывается в виде матрицы с N строками и $N + 1$ столбцами. Над строками выполняются некоторые операции до тех пор пока не получается единичная матрица в первых N строках и N столбцах. Тогда в последнем столбце матрицы оказываются искомые значения неизвестных. Теперь приведем *SIMD*-алгоритм в модели *CREW PRAM*, реализующий тот же самый подход. В нижеследующем обсуждении предполагается, что в ячейке памяти M_{ij} хранится коэффициент при j-ом неизвестном (j-ЫЙ столбец) в i-ом уравнении (i-ая строка).

Прежде, чем изложить параллельный алгоритм, приведем последовательный алгоритм. Процесс начинается с деления первой строки на ее первый элемент. Так, если первый элемент первой строки равен 5, то нужно разделить все элементы первой строки на 5. Затем последовательный алгоритм вычитает из всех остальных строчек первую строку, умноженную на первый элемент этих строчек. Например, если первый элемент второй строки равен 12, то из второй строки нужно вычесть первую, умноженную на 12.

Следующий алгоритм в модели *CREW PRAM* работает на $N * (N + 1)$ процессорах, каждый из которых обновляет один элемент матрицы. Как и в последовательном алгоритме, реализующем метод Гаусса - Жордана, этот параллельный алгоритм не занимается ошибками округления или матричными особенностями.

В этом алгоритме внешний цикл итерируется по всем неизвестным. Первый параллельный блок делит элементы текущей строки на требуемый коэффициент. Затем второй параллельный блок вычитает подходящее кратное текущей строки из всех остальных строк. В этом втором блоке внешний цикл обрабатывает те столбцы, которым еще требуется обработка, а внутренний цикл выполняет обработку всех уравнений.

Параллельный алгоритм решения системы N линейных уравнений с N неизвестными методом Гаусса - Жордана имеет следующий вид:

```
for x = 1 to N do
    Parallel Start
        for y = x to N + 1 do
            P[x, y] читает M[x, x] в factor, а M[x, y] в value
            P[x, y] вычисляет value / factor и записывает в M[x, y]
        end for y
    Parallel End
    Parallel Start
        for y = x to N + 1 do
            for z = 1 to N do
                if x /= z
                    P[z, y] читает M[z, y] в current,
                        M[z, x] в factor, а M[x, y] в value
                    P[x, y] вычисляет current – value / factor
                        и записывает результат в M[z, y]
                end if
            end for z
        end for y
    Parallel End
end for x
```

Параллельные алгоритмы на графах

При исследовании параллельных алгоритмов на графах будем представлять графы матрицами смежности. Мы займемся некоторыми замечательными связями между матричными операциями над матрицами смежности и свойствами соответствующих графов.

Параллельный алгоритм поиска кратчайшего пути

Определим матрицу смежности взвешенного графа следующим образом:

$$AdjMat[i,j] = \begin{cases} w_{ij} \text{ при } v_i v_j \in E, \\ 0 \quad \text{при } i = j, \\ \infty \quad \text{при } v_i v_j \notin E \end{cases} \qquad \text{для } i, j \in [1, N].$$

На рис. 10 изображен взвешенный граф и его матрица смежности. В матрице смежности представлены прямые пути между вершинами графа, т. е. пути, состоящие из одного ребра. Нас интересуют кратчайшие путипо графу, состоящие из произвольного числа ребер. Попробуем восстановить их по матрице смежности. В исходной матрице $A = A^1$ содержатся длины кратчайших путей из нуля или одного ребра. В матрице A^j будут записаны длины кратчайших путей из не более чем j ребер. Ясно, что в матрице A^{N-1} будут записаны длины всех кратчайших путей с произвольным числом ребер, потому что всякий путь из N и более ребер содержит цикл, а значит, кратчайший путь должен содержать не более $N - 1$ ребер.

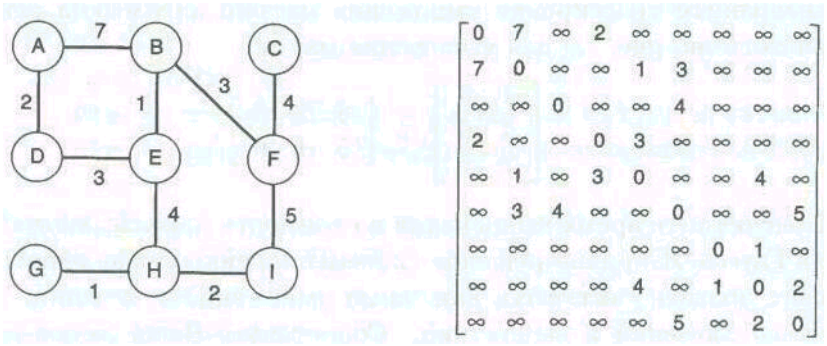


Рис. 10. Взвешенный граф и его матрица смежности

Подумаем, как можно построить матрицу A^2 по матрице A^1. Кратчайший путь из двух ребер между вершинами x и y проходит еще в точности через одну вершину. Например, всякий путь длины два между вершинами A и E проходит либо через вершину B, либо через вершину D. Сравнив сумму весов ребер AB и BE с суммой весов ребер AD и DE, видим, что путь через вершину D короче пути через вершину B. В общем случае, если посмотреть на сумму весов ребер $A*$ и $E*$, где $*$ пробегает все вершины от A до I (за исключением самих вершин A и E), то минимальное значение суммы весов будет давать длину кратчайшего пути из двух или менее ребер. Отсюда получаем

$$A_{ij}^2 = \min_{k \in V}(A_{ik}^1 + A_{kj}^1).$$

Применив эту процедуру к матрице смежности на рис. 10, получим матрицу, показанную на рис. 11.

$$\begin{bmatrix} 0 & 7 & \infty & 2 & 5 & 10 & \infty & \infty & \infty \\ 7 & 0 & 7 & 4 & 1 & 3 & \infty & 5 & 8 \\ \infty & 7 & 0 & \infty & \infty & 4 & \infty & \infty & 9 \\ 2 & 4 & \infty & 0 & 3 & \infty & \infty & 7 & \infty \\ 5 & 1 & \infty & 3 & 0 & 4 & 5 & 4 & 6 \\ 10 & 3 & 4 & \infty & 4 & 0 & \infty & 7 & 5 \\ \infty & \infty & \infty & \infty & 5 & \infty & 0 & 1 & 3 \\ \infty & 5 & \infty & 7 & 4 & 7 & 1 & 0 & 2 \\ \infty & 8 & 9 & \infty & 6 & 5 & 3 & 2 & 0 \end{bmatrix}$$

Рис. 11. Матрица A^2 для взвешенный граф с рис. 10

Матрицу A^3 можно построить по матрицам A^1 и A^2, заметив, что кратчайший путь из трех или менее ребер должен состоять из кратчайшего пути из двух или менее ребер, за которым следует кратчайший путь из одного или менее ребер и наоборот. Матрицу A^4 можно построить либо по матрицам A^1 и A^3, либо только по матрице A^2. Поэтому до конца добраться легче, вычисляя матрицы A^2, A^4, A^8, …, $A^{N'}$, где N' – первая степень двойки которая превышает число вершин, уменьшенное на 1. Все кратчайшие расстояния в графе с рис. 10 получим, подсчитав матрицу A^8.

Параллельный подсчет кратчайших расстояний в графе можно реализовать на основе этих матричных операций. Заменив в алгоритме умножения матриц сложение операцией взятия минимума, а умножение сложением, получим алгоритм, вычисляющий нужные нам матрицы. Применение модифицированного алгоритма матричного умножения к матрицам A^1 и A^1 даст матрицу A^2, а к матрицам A^2 и A^2 – матрицу A^4. Теперь параллельный алгоритм подсчета кратчайших расстояний превращается просто в параллельный алгоритм умножения матриц, поэтому анализ последнего применим и в данном случае.

Параллельный алгоритм поиска минимального остовного дерева

Напомним, что алгоритм Прима порождения минимального остовного дерева строит дерево постепенно, добавляя на каждом шаге вершину, соединенную с уже построенной частью дерева ребром минимальной длины. При этом вершины,

расположенные рядом с уже построенной частью дерева, образуют «кайму». Располагая большим числом процессоров, можно вместо этого рассматривать одновременно все вершины и при каждом проходе выбирать ближайшую из них.

Спроектируем алгоритм для p процессоров, предполагая, что $p < N$. Это означает, что каждый процессор будет отвечать за N / p вершин. Начнем построение минимального остовного дерева с одной из вершин. В начальный момент это ближайшая ко всем остальным вершина дерева, поскольку она единственная. При каждом проходе каждый процессор исследует каждую из вершин и выбирает из них ближайшую к вершинам дерева. Затем процессор передает информацию о выбранной вершине центральному процессору, который выбирает среди них вершину с абсолютно минимальным расстоянием до построенной части дерева. Эта вершина добавляется к дереву, а информация о ней передается остальным процессорам, и они обновляют свои данные о дереве. Процесс повторяется $N - 1$ раз пока к дереву не будут добавлены все вершины.

В следующем алгоритме множество вершин, за которые отвечает процессор P_j, обозначается через V_j, а через v_k обозначается вершина графа. В каждом из процессоров локально используется два массива. Элемент *closest*(v) первого из них содержит метку вершины построенной части минимального остовного дерева, ближайшей к вершине v, а элемент *distance* ($v[i]$, $v[j]$) – кратчайшее расстояние между вершинами v_i и v_j. Вот как выглядит этот алгоритм в модели *CREW PRAM*:

```
v[0] помечается как первая вершина дерева
Parallel Start
    for j = 1 to p do
        for каждой вершины в V[j] do
            занести v[0] в closest
        end for
    end for j
Parallel End
```

for j = 1 *to N* – 1 *do*

Parallel Start

for k = 1 *to p do*

P[*k*] вычисляет наименьшие расстояния от своих вершин до минимального остовного дерева

P[*k*] возвращает *distance*(*v*, *closest*(*v*)), *v* и closest(v)

end for k

Parallel End

Центральный процессор *P* находит наименьшее из найденных расстояний и добавляет соответствующую вершину *v* в минимальное остовное дерево

Центральный процессор *P* сообщает вновь добавленную вершину *v* всем остальным процессорам

Parallel Start

for k = 1 *to p do*

if v лежит в *V*[*k*] *then*

P[*k*] помечает, что *v* занесена в дерево

end if

Pk меняет значения *closest* и *distance* для всех вершин еще не занесенных в дерево, учитывая вновь добавленную в дерево вершину

end for k

Parallel End

end for j

Анализ

Первый цикл алгоритма устанавливает начальные значения. Его выполнение требует N / p шагов, поскольку количество начальных значений, устанавливаемых каждым процессором, пропорционально числу вершин, за которые он отвечает. Первый параллельный блок в главном цикле *for* всякий раз

выполняет $N / p - 1$ сравнений, ровно столько сравнений требует последовательный алгоритм поиска максимума или минимума. Следующий шаг состоит в том, чтобы выбрать минимальное расстояние из всех сообщенных p процессорами, на что уходит еще $p - 1$ сравнений. Шаг распределения значений в модели *CREW* занимает, как было показано, два цикла. В последнем параллельном блоке одно сравнение уходит на то, чтобы проверить, отвечает ли данный процессор за вновь добавленный узел, а исправление массивов *closest* и *distance* требует N / p операций. Итак, при одном проходе главного цикла обработки выполняется $2(N / p) + p + 1$ операций, а число проходов равно $N - 1$. Поэтому общее число операций в главном цикле $(N - 1) * (2 * (N / p) + p + 1)$, а общая сложность алгоритма равна $O(N^2 / p)$ при стоимости $p * O(N^2 / p)$, или $O(N^2)$. Как мы уже видели в других случаях, оптимальность достигается при числе процессоров около $N / \log N$.